

Statement

Consider a standard binary heap, as described e.g. in [this Wikipedia article revision \(https://en.wikipedia.org/w/index.php?title=Binary_heap&oldid=1292888031\)](https://en.wikipedia.org/w/index.php?title=Binary_heap&oldid=1292888031).

We have a tree with n empty nodes that has the shape of a n -element binary heap. We are going to place the values 1 through n into the nodes in such a way that they form a max-heap.

Let c_x be the number of nodes that can contain the value x .

Determine the values c_x . Then, calculate and output the value $\sum_{i=1}^n i \cdot c_i$.

Input format

The first line of each input file contains the number t of test cases. The specified number of test cases follows, one after another.

Each test case consists of a single line with a single positive integer n . Each test has $n \leq 2\,500\,000$.

Output format

For each test case output a single line with a single integer. Ideally, that integer should be the value of the sum defined above.

You may assume that the correct answer for each test case fits into a signed 64-bit integer variable.

Subproblem H1 (up to 100 points, public)

Input file: [H1.in \(/static/online2025/H1.f3fb96d7f6eb88b6463bdcda62eb610f.in\)](/static/online2025/H1.f3fb96d7f6eb88b6463bdcda62eb610f.in).

There is a single input file for this problem. You may submit output files in which some of the outputs are incorrect and still receive a partial score. (E.g., you may output -1 instead of the correct answer for tests that you don't want to solve.)

Your output file **must contain** exactly t integers. If it contains anything else, you will score zero.

Your score will be computed by adding up the following:

- +7 points if you correctly solve all tests with $n \leq 10$
- +20 points if you correctly solve all tests with $n \leq 1024$ where $n + 1$ is a power of 2
- +30 points if you correctly solve all tests with $n \leq 1024$
- +43 points if you correctly solve all tests

Example

input	output
3 2 3 4	3 9 18

For $n = 2$ the value 2 must be in the root and the value 1 must be in its only child. Each value has a single position where it can appear, hence $c_1 = c_2 = 1$.

For $n = 3$ the value 3 must be in the root and the other two values in the leaves, in either order. Hence, $c_3 = 1$ but $c_1 = c_2 = 2$. The answer is $1 \cdot 2 + 2 \cdot 2 + 3 \cdot 1$.

For $n = 4$ we have $c_1 = 2$, $c_2 = 3$, $c_3 = 2$, and $c_4 = 1$.